Homework: Physics-Informed Neural Networks for a Two-Dimensional Poisson Equation

Numerical Methods for Partial Differential Equations

Homework Problem

In this homework, you will implement a Physics-Informed Neural Network (PINN) for solving a two-dimensional Poisson equation. The model problem is based on the Poisson-equation example discussed in Baty's PINN paper.

Let

$$\Omega = (0,1) \times (0,1).$$

Consider the Poisson equation

$$\Delta u(x,y) = f(x,y), \qquad (x,y) \in \Omega,$$

where

$$\Delta u = u_{xx} + u_{yy}.$$

The exact solution is chosen as

$$u_{\text{ex}}(x,y) = e^{xy}.$$

The boundary condition is the Dirichlet condition

$$u(x,y) = u_{\text{ex}}(x,y), \qquad (x,y) \in \partial\Omega.$$

Part 1. Derivation of the Source Term

(1) Starting from

$$u_{\text{ex}}(x,y) = e^{xy},$$

compute

$$u_x, \qquad u_y, \qquad u_{xx}, \qquad u_{yy}.$$

(2) Show that

$$u_{xx} = y^2 e^{xy}, \qquad u_{yy} = x^2 e^{xy}.$$

(3) Hence derive the source term

$$f(x,y) = \Delta u_{\text{ex}}(x,y).$$

(4) Verify that

$$f(x,y) = e^{xy}(x^2 + y^2).$$

Therefore, the complete boundary-value problem is

$$\boxed{\begin{aligned} u_{xx} + u_{yy} &= e^{xy}(x^2+y^2), & (x,y) &\in (0,1)^2, \\ u(x,y) &= e^{xy}, & (x,y) &\in \partial\Omega. \end{aligned}}$$

Part 2. PINN Approximation

Let a fully connected neural network approximate the solution:

$$u_\theta(x, y) \approx u(x, y),$$

where θ denotes all trainable weights and biases of the neural network.

The input of the network is

$$(x, y) \in \mathbb{R}^2,$$

and the output is

$$u_\theta(x, y) \in \mathbb{R}.$$

Use a neural network with the following structure:

$$2 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow 1.$$

That is, use

$$6$$

hidden layers, with

$$20$$

neurons in each hidden layer.

Use the hyperbolic tangent activation function:

$$\sigma(z) = \tanh(z).$$

Part 3. PDE Residual

Define the PINN residual

$$\mathcal{R}_\theta(x, y) = \frac{\partial^2 u_\theta}{\partial x^2}(x, y) + \frac{\partial^2 u_\theta}{\partial y^2}(x, y) - f(x, y).$$

For this problem,

$$f(x, y) = e^{xy}(x^2 + y^2).$$

Therefore,

$$\boxed{\mathcal{R}_\theta(x, y) = u_{\theta,xx}(x, y) + u_{\theta,yy}(x, y) - e^{xy}(x^2 + y^2).}$$

In your implementation, compute $u_{\theta,xx}$ and $u_{\theta,yy}$ using automatic differentiation.

Part 4. Training Points

Use two types of training points.

Interior Collocation Points

Choose

$$N_c = 400$$

collocation points in the interior of the domain:

$$(x_i^c, y_i^c) \in \Omega, \qquad i = 1, \dots, N_c.$$

You may use either uniform random sampling or Latin hypercube sampling.

Boundary Data Points

Choose

$$N_b = 120$$

boundary points on $\partial\Omega$, for example 30 points on each side:

$$x = 0, \qquad x = 1, \qquad y = 0, \qquad y = 1.$$

At the boundary points, the exact Dirichlet data are

$$u_{\text{ex}}(x, y) = e^{xy}.$$

Part 5. Loss Function

The PINN loss consists of two parts:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \mathcal{L}_{\text{BC}}(\theta).$$

The PDE loss is

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} |\mathcal{R}_\theta(x_i^c, y_i^c)|^2 .$$

That is,

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} \left| u_{\theta,xx}(x_i^c, y_i^c) + u_{\theta,yy}(x_i^c, y_i^c) - e^{x_i^c y_i^c}\left((x_i^c)^2 + (y_i^c)^2\right)\right|^2 .$$

The boundary loss is

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_b}\sum_{j=1}^{N_b} \left| u_\theta(x_j^b, y_j^b) - u_{\text{ex}}(x_j^b, y_j^b)\right|^2 .$$

Since

$$u_{\text{ex}}(x, y) = e^{xy},$$

we have

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_b}\sum_{j=1}^{N_b} \left| u_\theta(x_j^b, y_j^b) - e^{x_j^b y_j^b}\right|^2 .$$

Thus the full training loss is

$$\boxed{\begin{aligned}\mathcal{L}(\theta) = \frac{1}{N_c}\sum_{i=1}^{N_c} &\left| u_{\theta,xx}(x_i^c, y_i^c) + u_{\theta,yy}(x_i^c, y_i^c) - e^{x_i^c y_i^c}\left((x_i^c)^2 + (y_i^c)^2\right)\right|^2 \\ &+ \frac{1}{N_b}\sum_{j=1}^{N_b} \left| u_\theta(x_j^b, y_j^b) - e^{x_j^b y_j^b}\right|^2 .\end{aligned}}$$

Part 6. Optimizer and Training Parameters

Use the Adam optimizer.

Use the learning rate

$$\eta = 3 \times 10^{-4}.$$

Train the network for

$$60,000$$

epochs.

For a quick test, you may first use

$$5,000$$

epochs, but the final submitted result should use a sufficiently large number of epochs.

Part 7. Numerical Experiments

After training the PINN, evaluate the solution on a uniform grid:

$$100 \times 100$$

points in $[0,1] \times [0,1]$.

Let

$$u_\theta(x_i, y_j)$$

be the predicted PINN solution and

$$u_{\text{ex}}(x_i, y_j) = e^{x_i y_j}$$

be the exact solution.

Compute the pointwise absolute error

$$E(x_i, y_j) = \left| u_\theta(x_i, y_j) - u_{\text{ex}}(x_i, y_j) \right|.$$

Report the maximum absolute error

$$E_\infty = \max_{i,j} \left| u_\theta(x_i, y_j) - u_{\text{ex}}(x_i, y_j) \right|.$$

Also report the mean absolute error

$$E_{\text{mean}} = \frac{1}{N} \sum_{i,j} \left| u_\theta(x_i, y_j) - u_{\text{ex}}(x_i, y_j) \right|,$$

where N is the total number of grid points.

Part 8. Required Plots

Submit the following plots.

(1) The boundary data points and interior collocation points.

(2) The loss history:

$$\mathcal{L}_{\text{PDE}}, \qquad \mathcal{L}_{\text{BC}}, \qquad \mathcal{L}.$$

Use a logarithmic scale for the vertical axis.

(3) The PINN-predicted solution $u_\theta(x, y)$.

(4) The exact solution

$$u_{\text{ex}}(x, y) = e^{xy}.$$

(5) The absolute error

$$|u_\theta(x, y) - u_{\text{ex}}(x, y)|.$$

(6) A comparison plot of the exact and predicted solutions along the line

$$y = 0.5.$$

That is, plot

$$u_\theta(x, 0.5)$$

and

$$u_{\text{ex}}(x, 0.5) = e^{0.5x}$$

on the same figure.

Part 9. Discussion Questions

Answer the following questions.

(1) Why does the PINN loss contain both a PDE residual loss and a boundary-condition loss?

(2) What happens if the weight of the boundary loss is too small?

(3) What happens if the number of collocation points is too small?

(4) Why is automatic differentiation useful in PINNs?

(5) Compare the PINN approach with a standard finite difference or finite element method for this Poisson problem. What are the advantages and disadvantages?

Submission Requirements

Submit the following items.

(1) A short derivation of the source term $f(x, y)$.

(2) A clear description of your PINN architecture.

(3) The loss function used in your implementation.

(4) The plots listed in Part 8.

(5) A table containing

$$E_\infty$$

and

$$E_{\text{mean}}.$$

(6) Your answers to the discussion questions.

(7) Your full Python code.

Suggested Error Table Format

N_c	N_b	Epochs	E_∞	E_{mean}
400	120	5000		
400	120	60000		

Remark

This problem is designed to illustrate the basic PINN methodology for an elliptic PDE. Since the exact solution is known, you can directly compare the PINN prediction with the exact solution and study the effect of training points, network size, and training epochs.